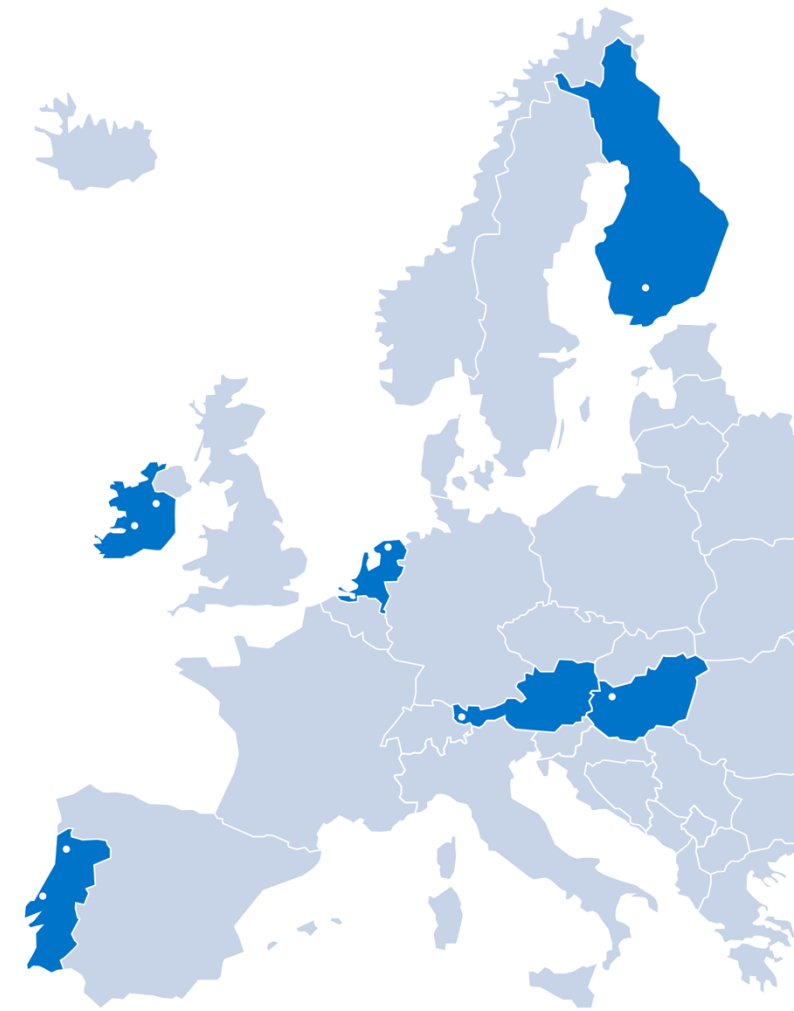


# IOT PENTEST

Jeroen Pijpker [jeroen.pijpker@nhlstenden.com](mailto:jeroen.pijpker@nhlstenden.com)

Nuno Lopes [nlopes@ipca.pt](mailto:nlopes@ipca.pt)



# IoT Pen Testing

- IoT penetration testing is a process of strengthening the IoT device security by finding existing security loopholes in the device and implementing proper security controls.
- Penetration testing of an IoT device involves testing the API, application, authentication policy, open ports, unencrypted information, and unencrypted communication between two end points.

# Steps in IoT Pen Testing (CEH EC-Council)

- Step 1: Discover IoT Devices
- Step 2: Hardware Analysis
- Step 3: Firmware and OS Analysis
- Step 4: Wireless Protocol Analysis
- Step 5: Mobile Application Testing
- Step 6: Web Application Testing
- Step 7: Cloud Services Testing
- Step 8: Document all the Findings!!!!

# Example Pentest reports

- IoT Offensive Security Penetration Testing  
Hacking a Smart Robot Vacuum Cleaner  
<https://kth.diva-portal.org/smash/get/diva2:1450590/FULLTEXT01.pdf>
- Testing IoT Security: The Case Study of an IP Camera  
[https://www.researchgate.net/publication/342184780 Testing IoT Security The Case Study of an IP Camera](https://www.researchgate.net/publication/342184780_Testing_IoT_Security_The_Case_Study_of_an_IP_Camera)
- Hacking IoT: A Case Study on Baby Monitor Exposures and Vulnerabilities  
<https://www.rapid7.com/globalassets/external/docs/Hacking-IoT-A-Case-Study-on-Baby-Monitor-Exposures-and-Vulnerabilities.pdf>



# THREAT MODELLING PROCESS

# Threat Modelling

Threat modelling is a description of a collection of security aspects, a set of plausible attacks which are able to affect the performance of any computer system.

This methodology allows security experts to identify security risks, and develop countermeasures in the design, coding, and testing phases.

Swiderski, F., Snyder, W.: Threat modeling. Microsoft Press Redmond, WA, USA (2004)

# Threat Modelling

- The goal of threat modelling is commonly to find security flaws and understand security requirements in order to engineer and deploy better products.
- That being said, it may also be used to find flaws and weaknesses in order to attempt to exploit them as an ethical hacker.
- Examples of Threat Modelling Frameworks:
  - STRIDE: *Threat classification model*
  - DREAD: *Risk rating model*
  - Microsoft Threat Modelling Process

# STRIDE

| Threat                         | Property        | Definition                                               | Example                                                                                                                             |
|--------------------------------|-----------------|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>S</b> poofing               | Authentication  | Impersonating something or someone else.                 | Pretending to be any of billg, xbox.com or a system update                                                                          |
| <b>T</b> ampering              | Integrity       | Modifying data or code                                   | Modifying a game config file on disk, or a packet as it traverses the network                                                       |
| <b>R</b> epudiation            | Non-repudiation | Claiming to have not performed an action                 | "I didn't cheat!"                                                                                                                   |
| <b>I</b> nformation Disclosure | Confidentiality | Exposing information to someone not authorized to see it | Reading key material from an app                                                                                                    |
| <b>D</b> enial of Service      | Availability    | Deny or degrade service to users                         | Crashing the web site, sending a packet and absorbing seconds of CPU time, or routing packets into a black hole                     |
| <b>E</b> levation of Privilege | Authorization   | Gain capabilities without proper authorization           | Allowing a remote internet user to run commands is the classic example, but running kernel code from lower trust levels is also EoP |

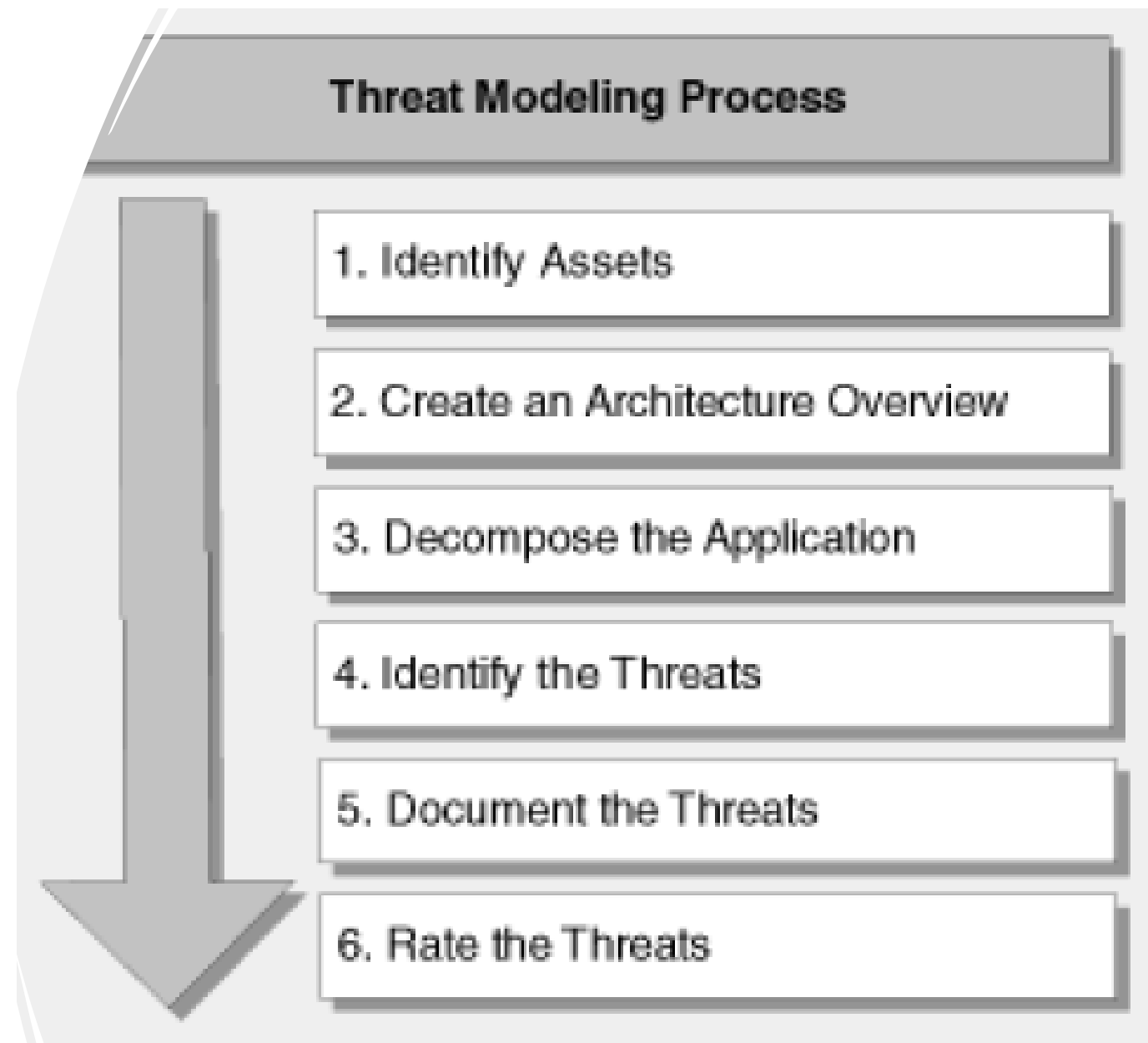
# DREAD

- The acronym DREAD is a risk rating system and stands for Damage Potential, Reproducibility, Exploitability, Affected users, and Discoverability which are rating categories.
- Each threat is given a rating in each of the categories from 1 to 3 and the sum of these ratings are calculated to give each threat a risk score. The points are given based on intuition and experience, and the score is used to give a guideline on which threats are the most important to focus on.
- DREAD methodology is used to rate, compare and prioritize the severity of risk presented by each threat that is classified using STRIDE.
- **DREAD Risk = (Damage + Reproducibility + Exploitability + Affected Users + Discoverability) / 5.** The higher the number means more serious the risk is.

# Microsoft Threat Modeling Process:

---

A methodology that can be used to threat model is a six-stage process proposed by Microsoft.



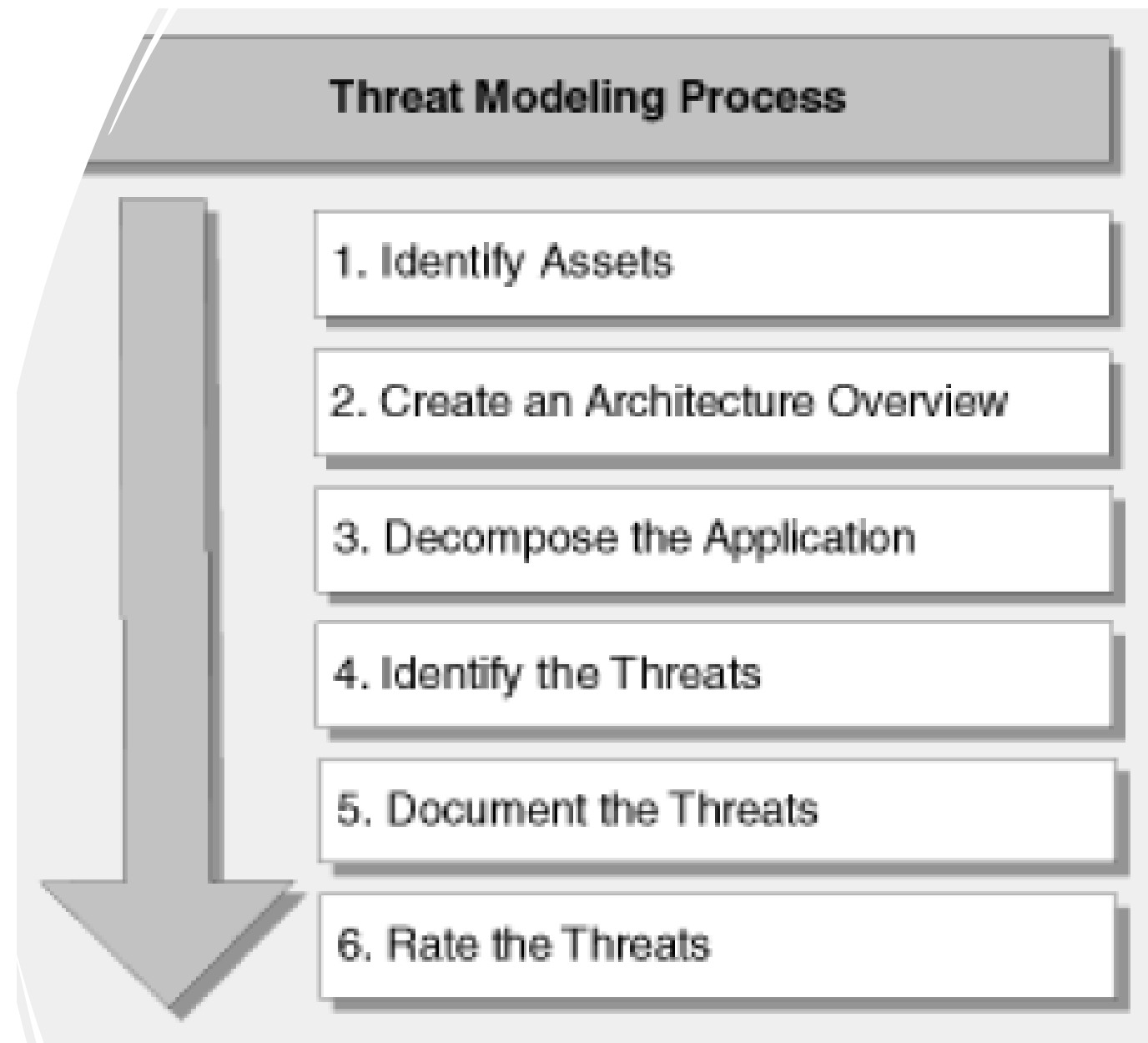
The six stages of the Microsoft threat modeling process<sup>11</sup>.

# Microsoft Threat Modeling Process:

---

First stage:

- Identify assets is simply to identify which assets, which here refers to some resource of value, that the system should protect.



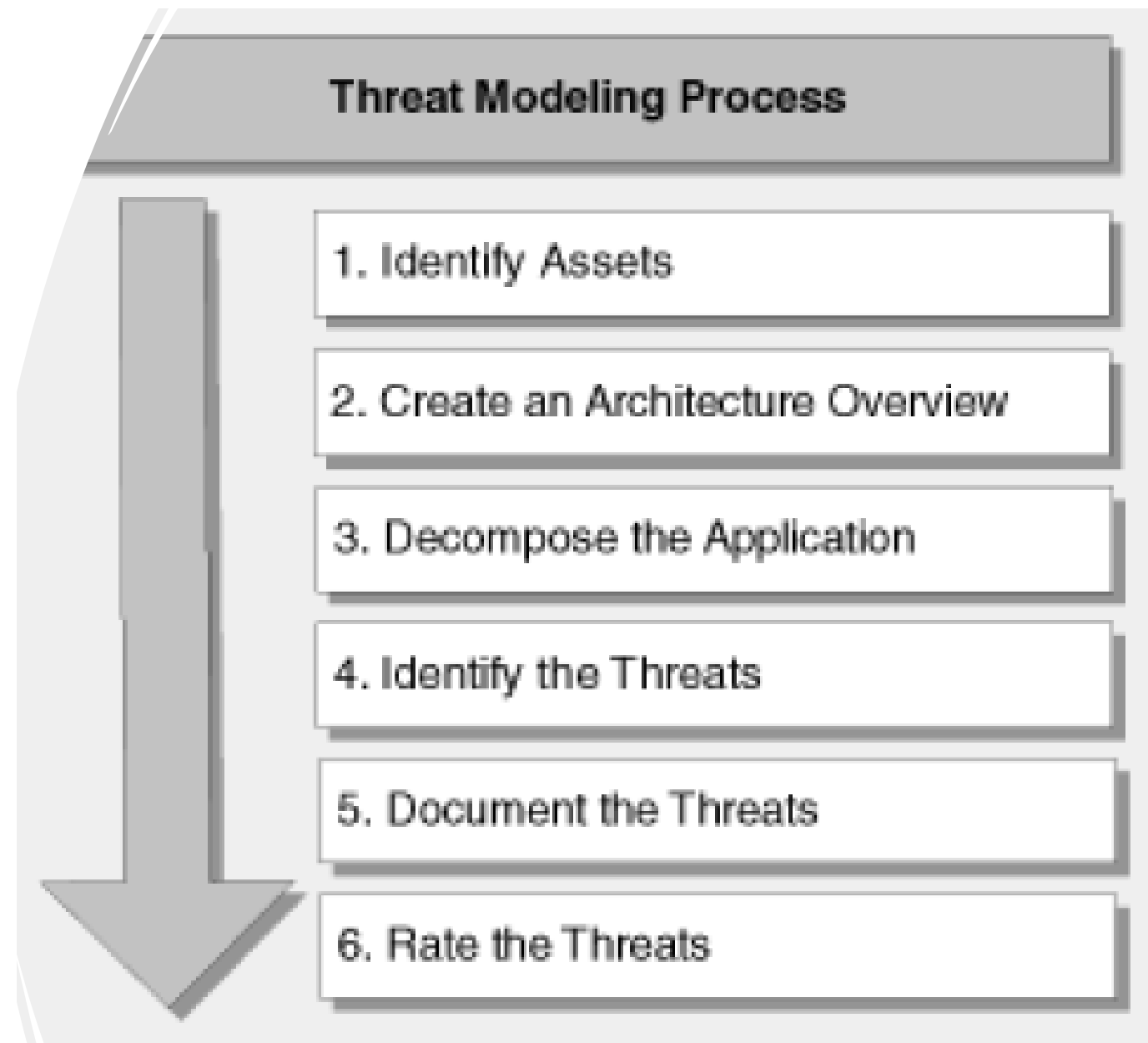
The six stages of the Microsoft threat modeling process<sup>11</sup>.

# Microsoft Threat Modeling Process:

---

Second stage:

- Create an architecture overview consists of using diagrams to model the architecture of the system and its information flows.



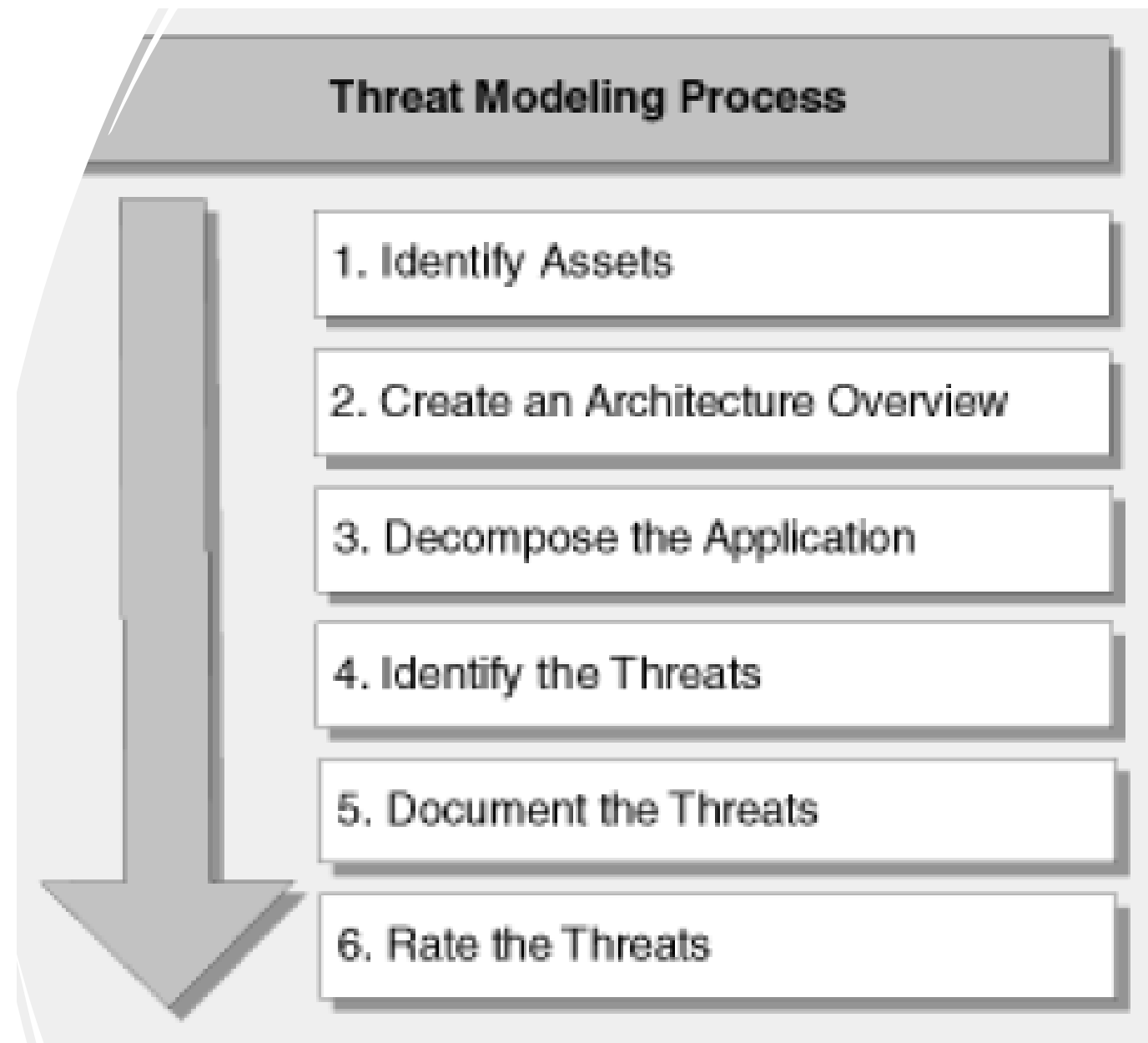
The six stages of the Microsoft threat modeling process<sup>11</sup>.

# Microsoft Threat Modeling Process:

---

Third stage:

- Decompose the application/device is performed by decomposing the architecture of the application/device in order to identify vulnerabilities in the design.



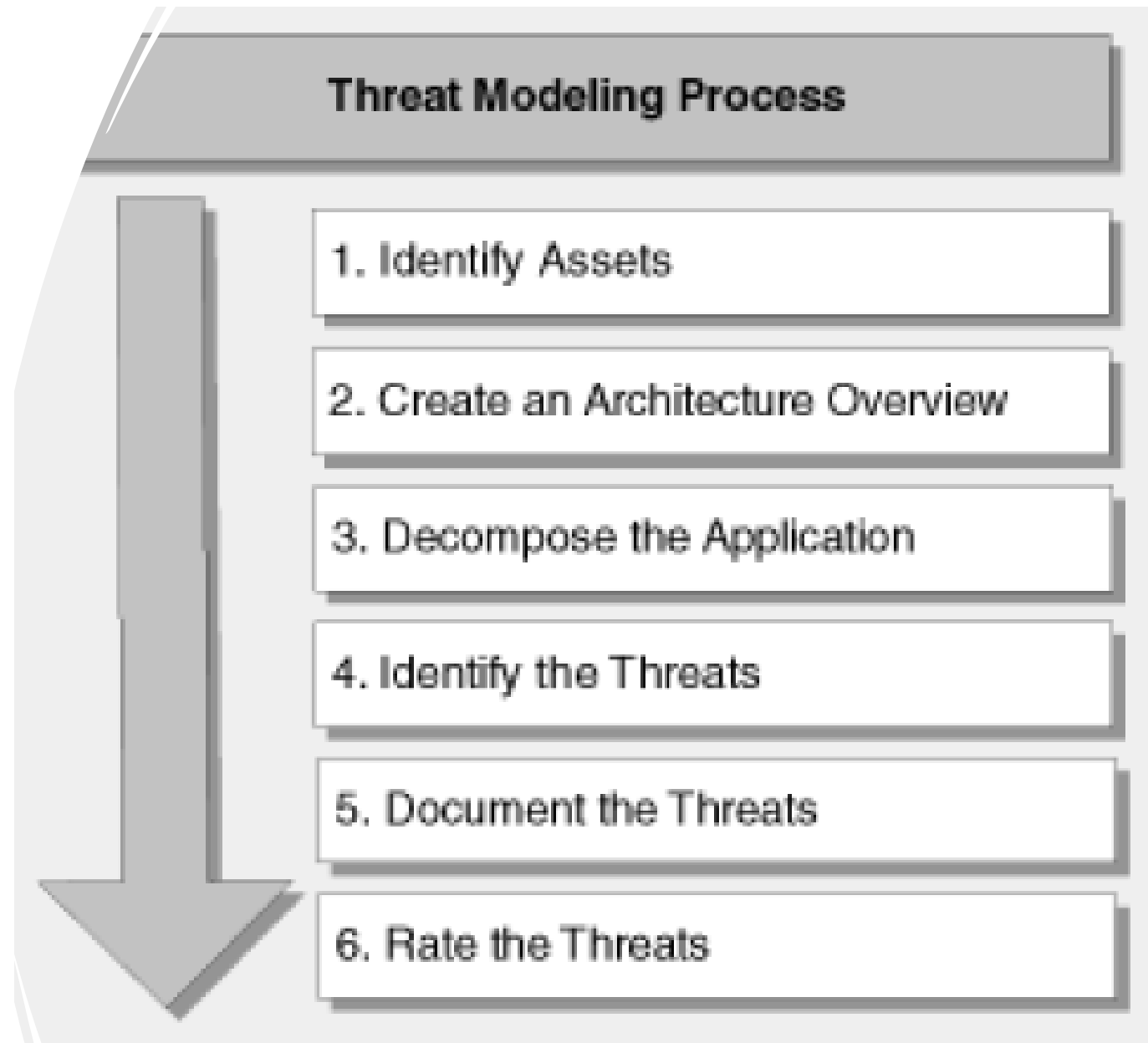
The six stages of the Microsoft threat modeling process<sup>11</sup>.

# Microsoft Threat Modeling Process:

---

Fourth stage:

- Identify the threats is to, with the acquired knowledge of the architecture and the vulnerabilities, identify the threats - here STRIDE could be of value.



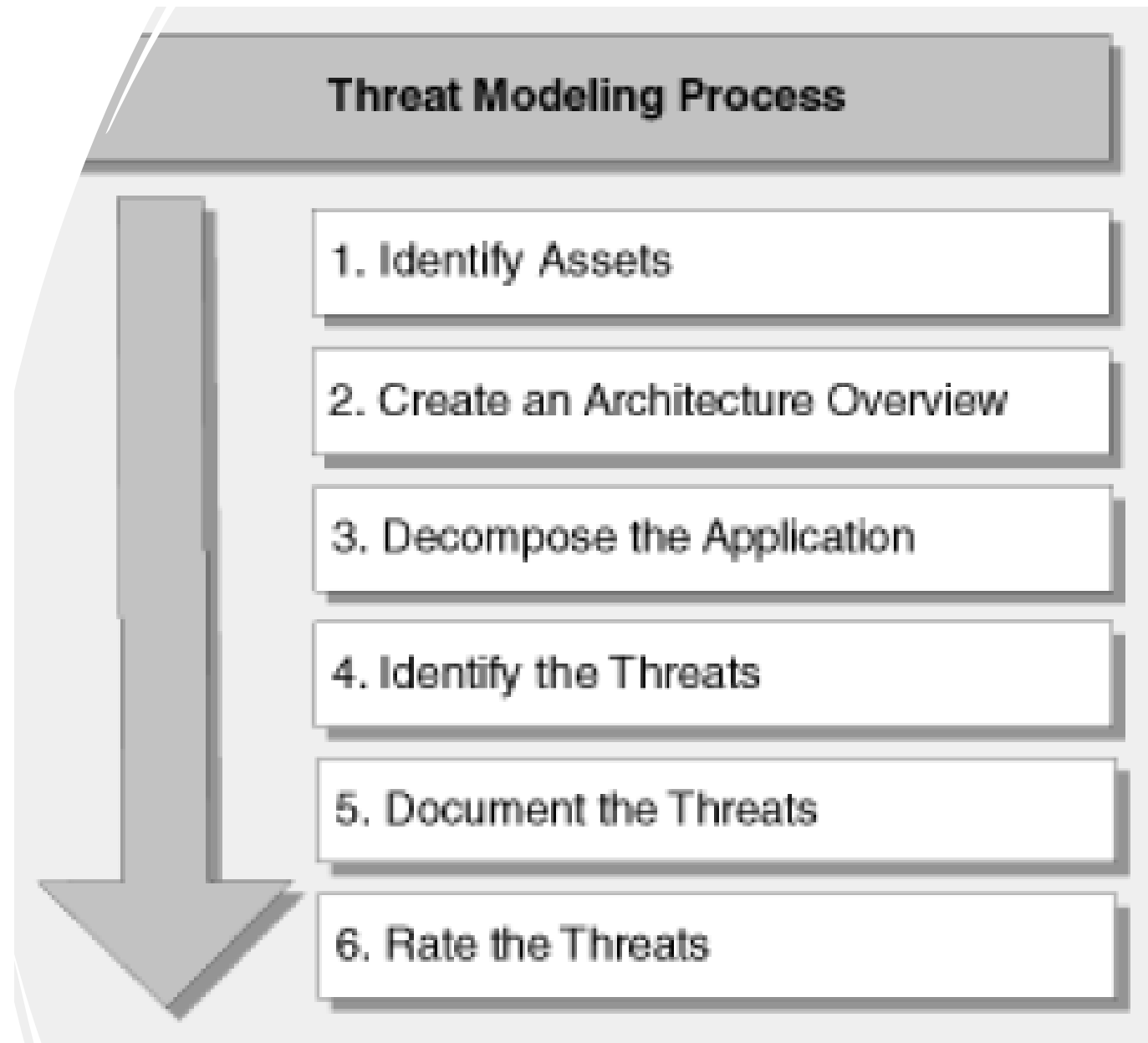
The six stages of the Microsoft threat modeling process<sup>11</sup>.

# Microsoft Threat Modeling Process:

---

Fifth stage:

- Document the threats is to document the threats using a common threat template the defines attributes for each of the threats.



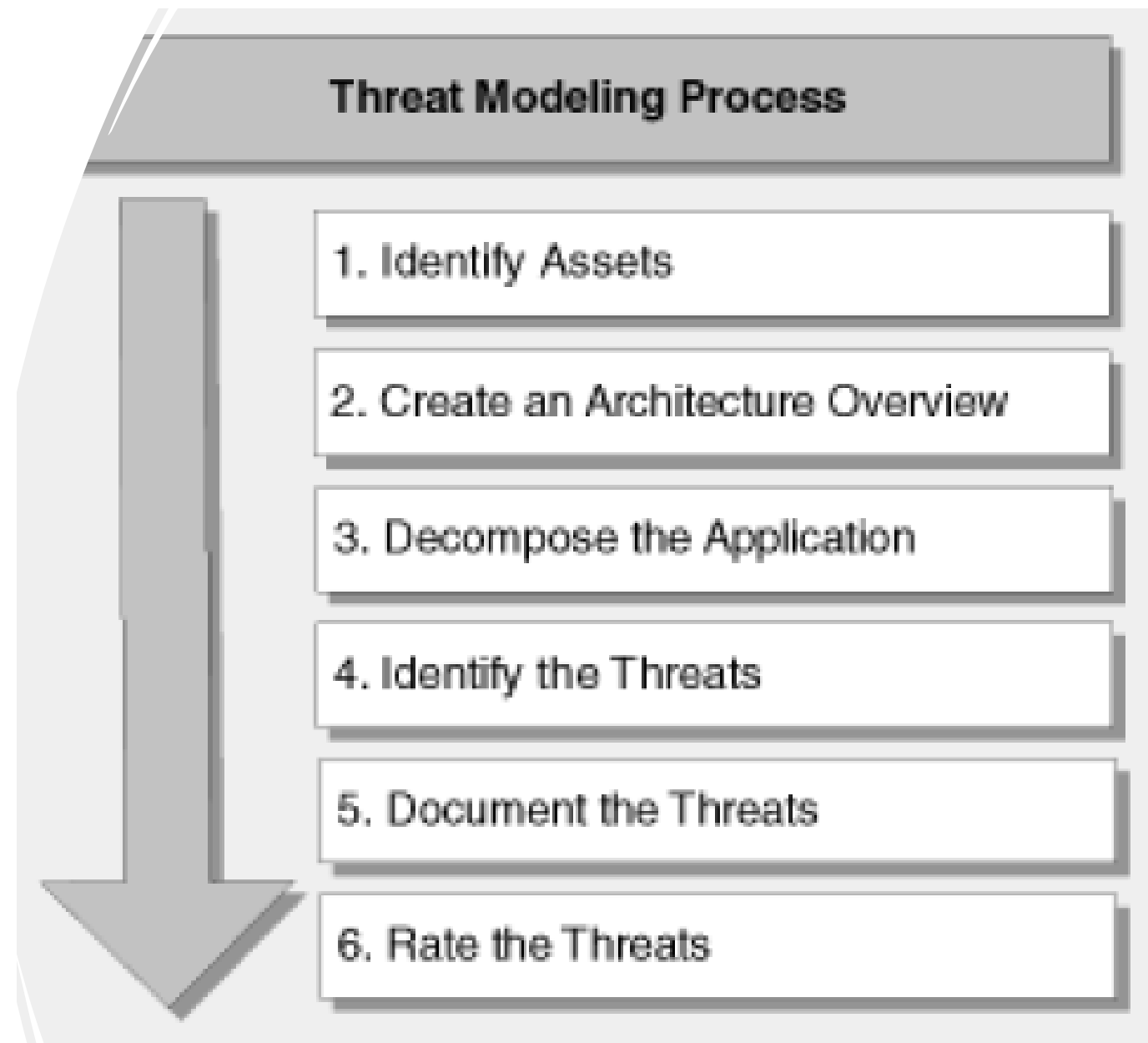
The six stages of the Microsoft threat modeling process<sup>11</sup>.

# Microsoft Threat Modeling Process:

---

Sixth stage:

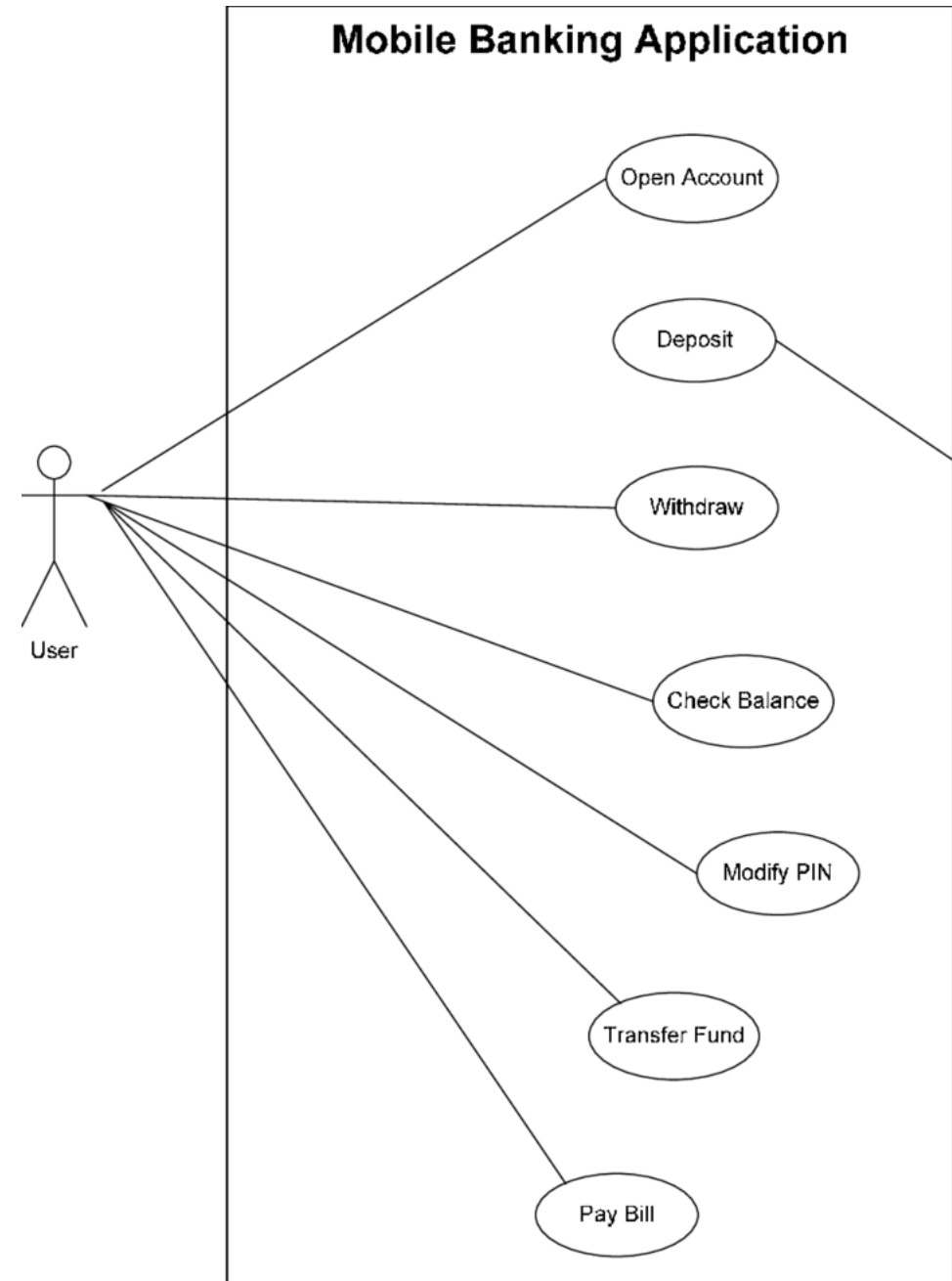
- Rate the threats can be performed by using DREAD to rate the threats in order to find out which threats to prioritize.



The six stages of the Microsoft threat modeling process<sup>11</sup>.

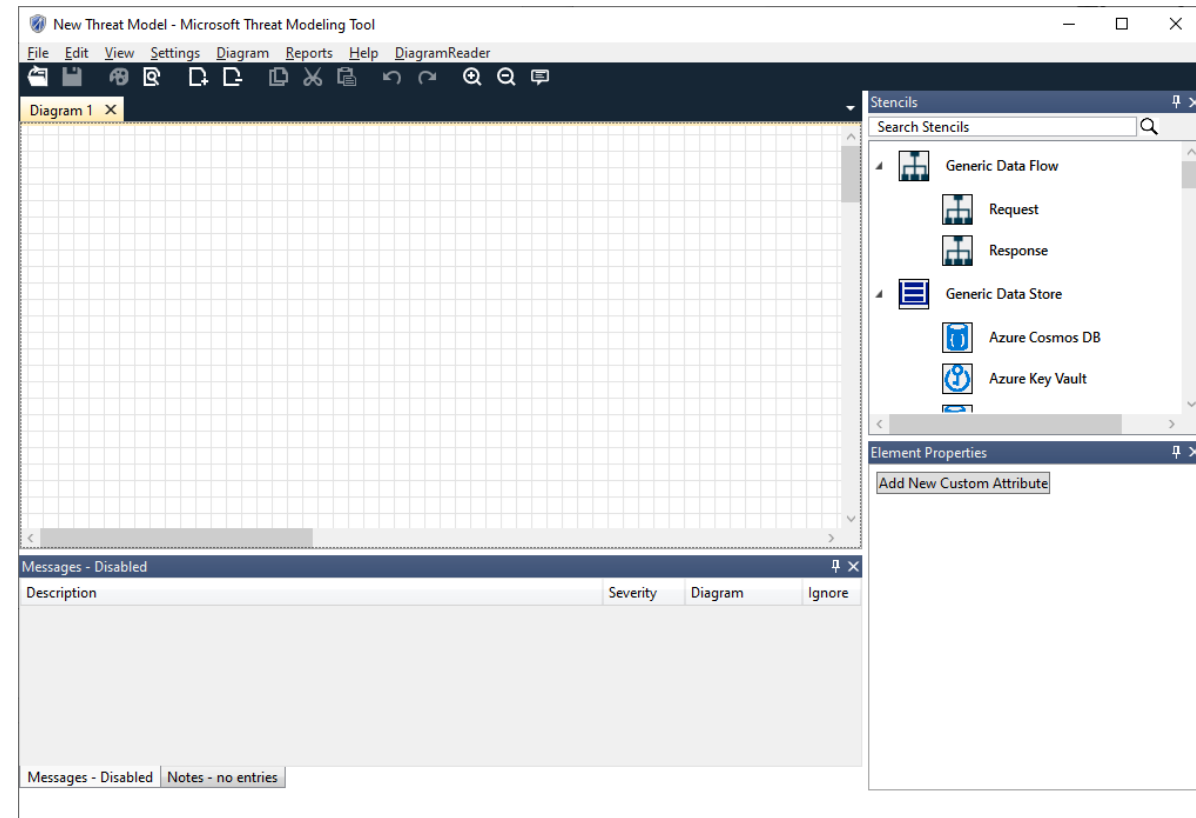
# Microsoft Threat Modeling Process:

**Use Cases:** Before creating an architecture diagram in the second stage of the Microsoft threat modelling process, an understanding of what the system does should be acquired. This could be done by creating so called use cases, which are examples of how a user could utilize the system. Doing this could also be helpful in working out how the system could be misused.

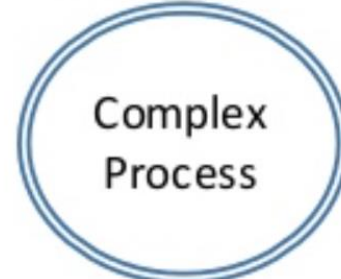
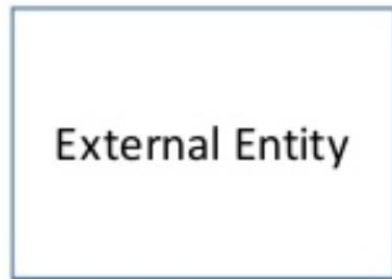


# Data Flow Diagram (DFD)

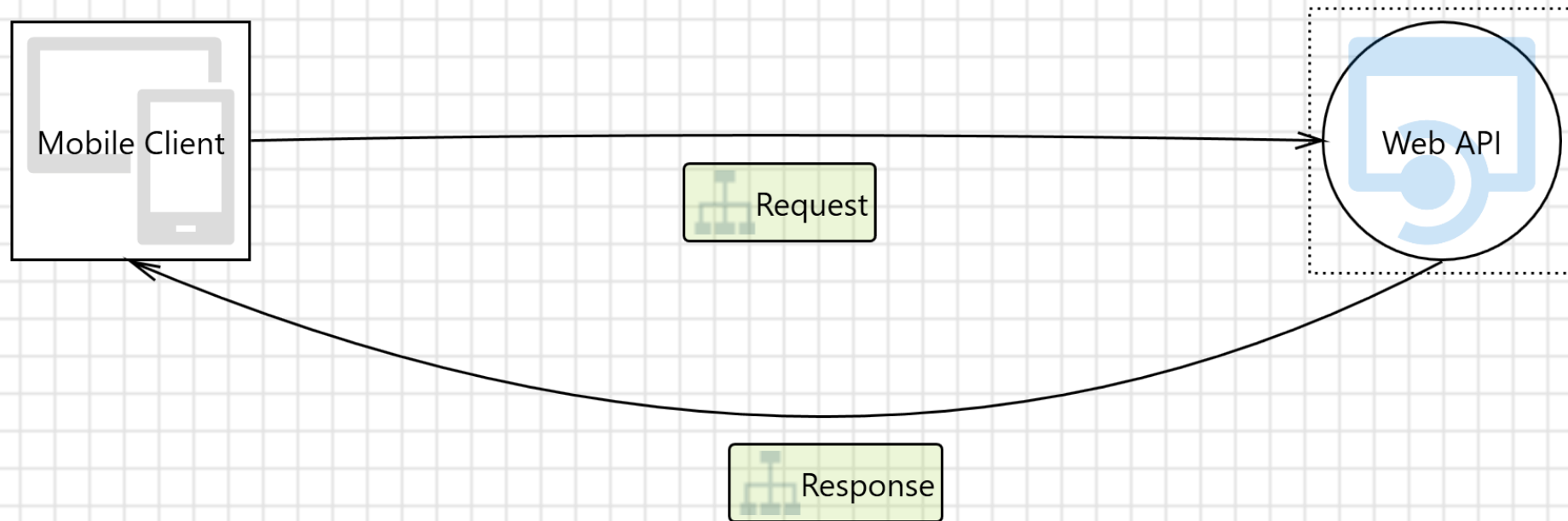
- Can help to understand the system
- Microsoft tool <https://aka.ms/threatmodelingtool>



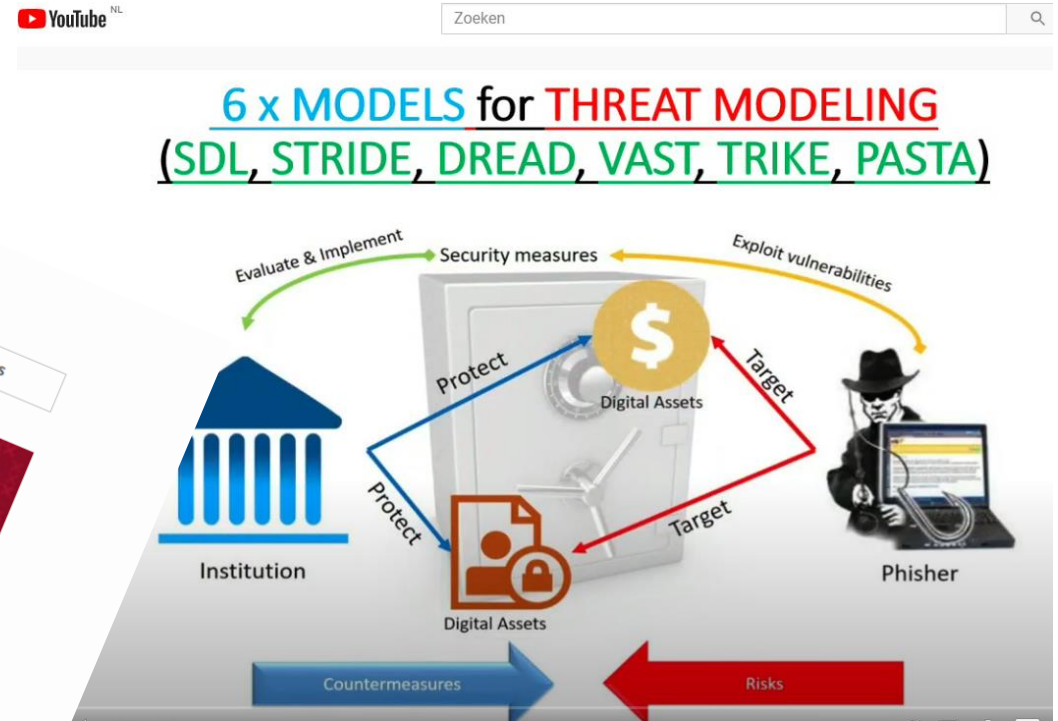
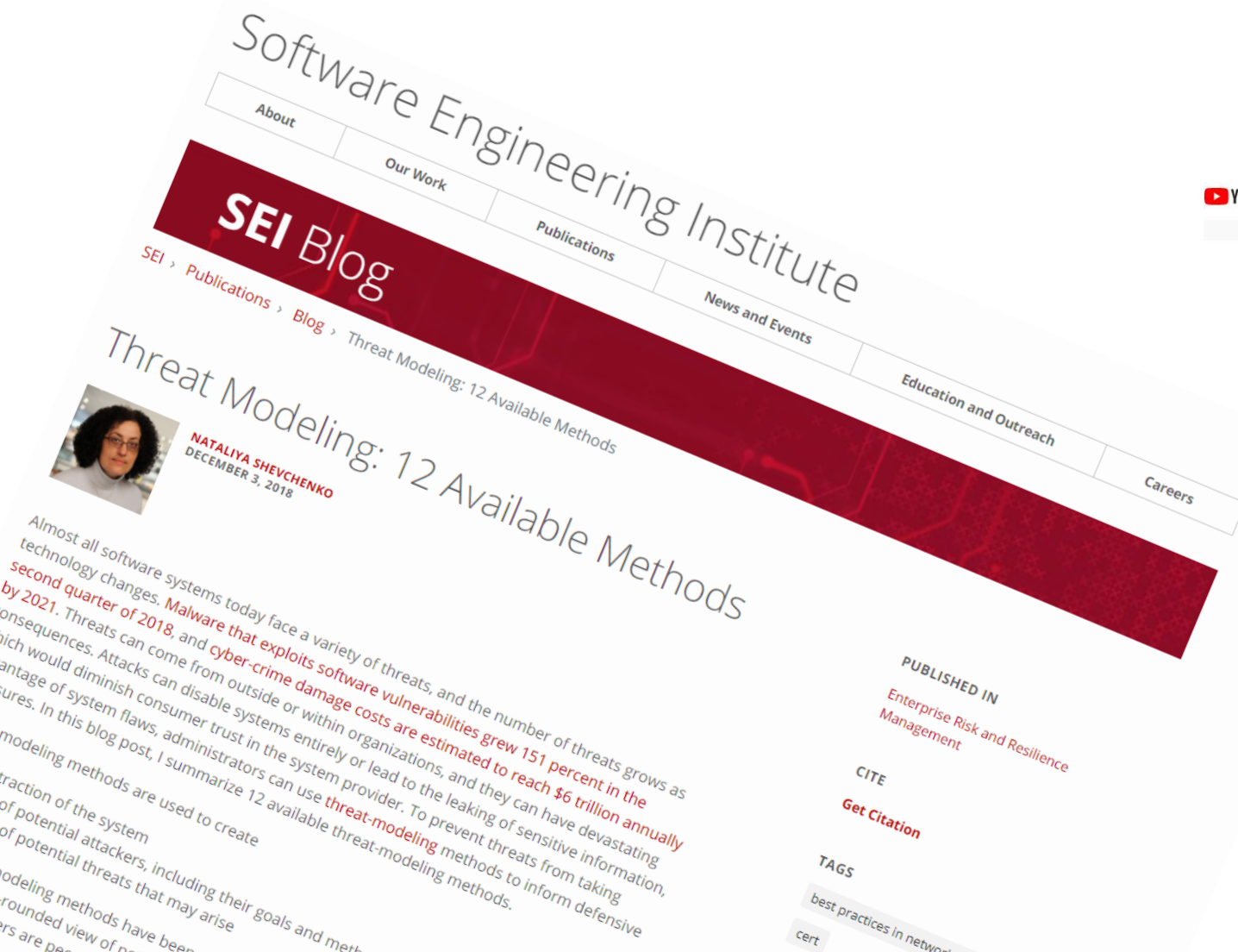
# Data Flow Diagram: Symbols



# Data Flow Diagram: Simple example



And there a lot more models for Threat Modelling



# Pen-testing IoT Devices for Vulnerabilities

---

So the question is what can we pentest?

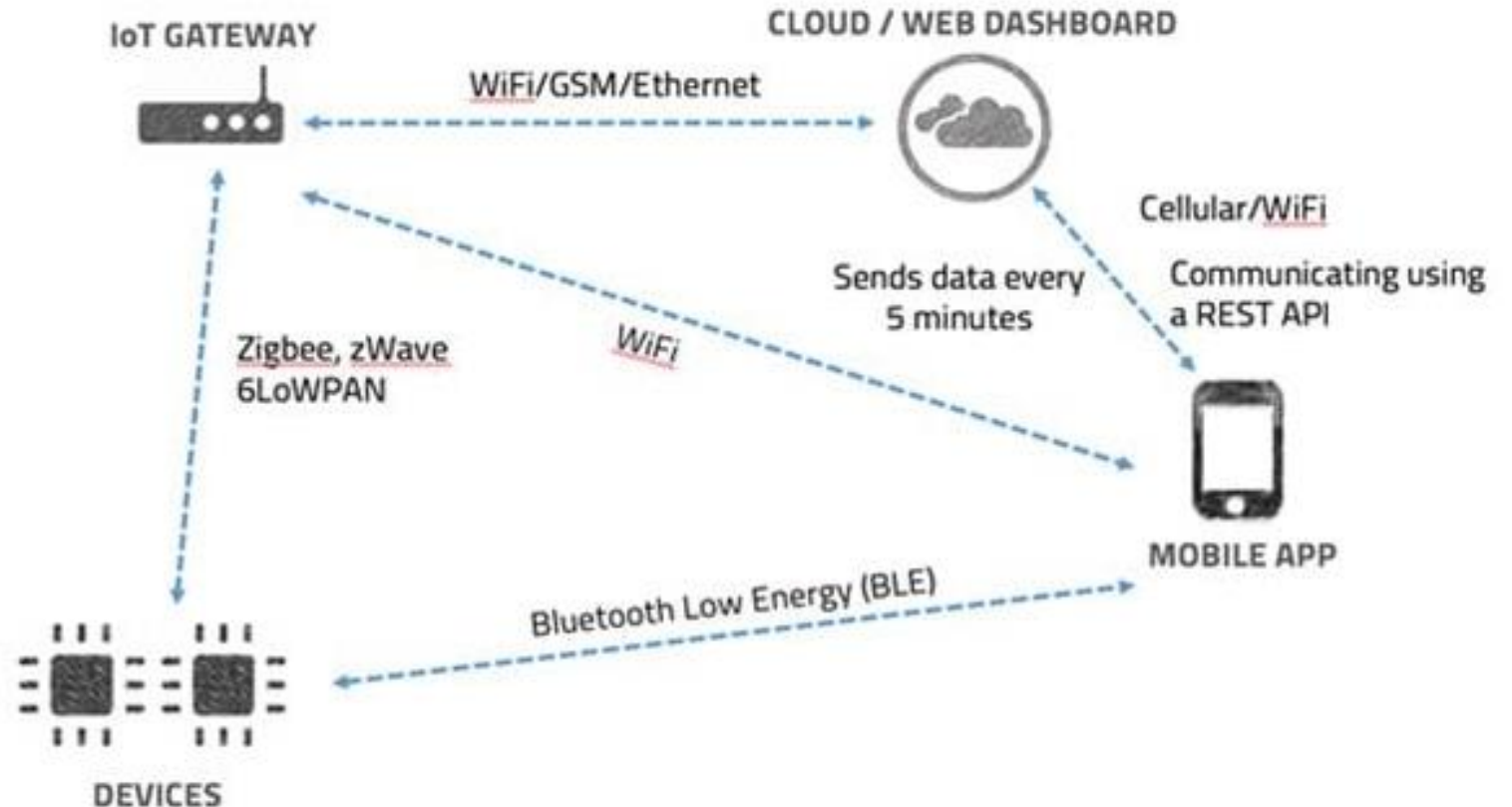
<https://www.cloudsek.com/blog/pentesting-iot-devices-for-vulnerabilities>



# The attack vectors

---

- Hardware
- Firmware
- Network
- Wireless Communications
- Mobile and web applications
- Cloud API's

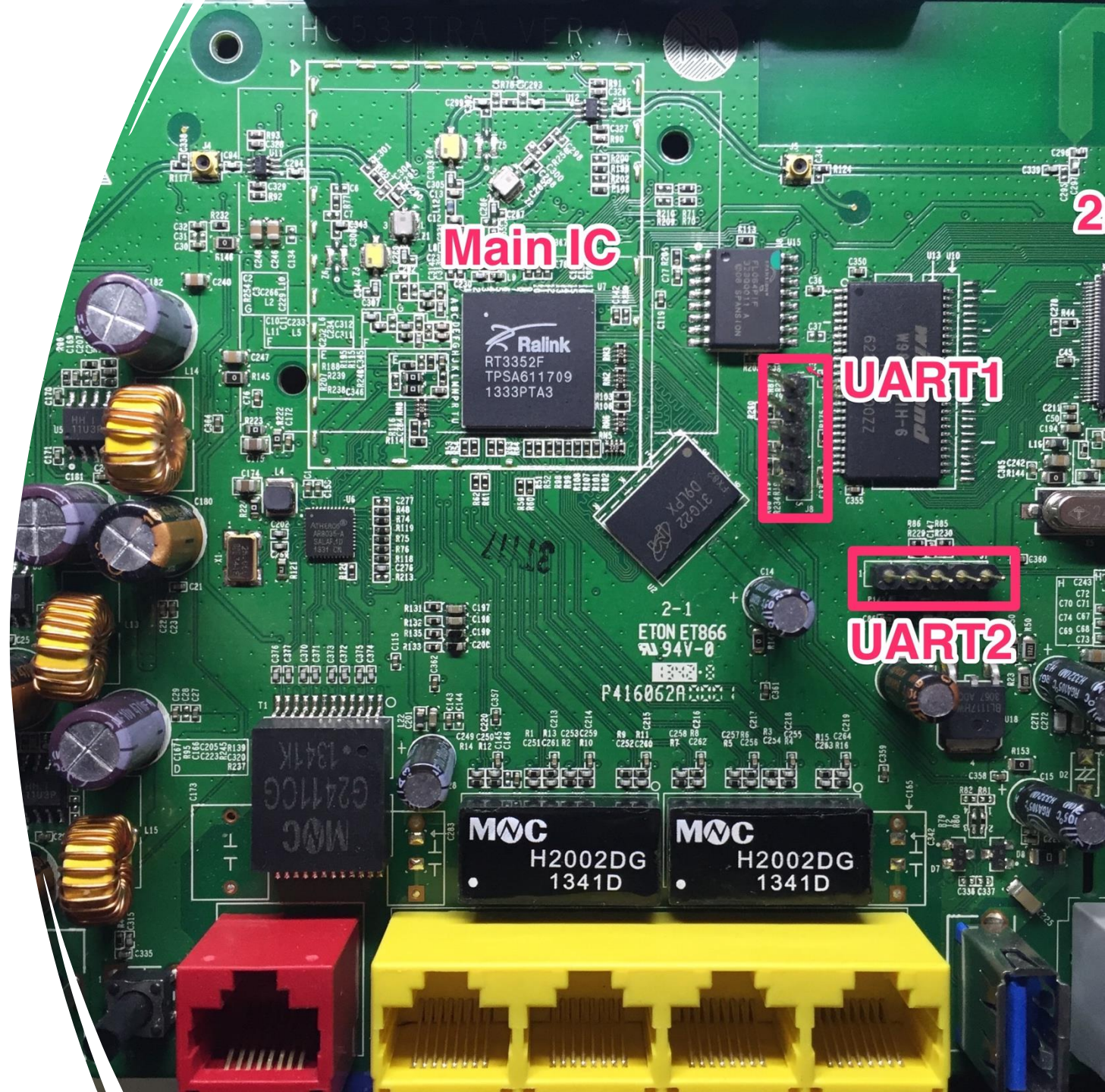


# IoT Device hardware pentest

- Internal communications protocols like UART, I2C, SPI...
- Open ports
- JTAG debugging
- Extracting firmware from EEPROM or FLASH memory
- Firmware download reverse engineering
- Tampering

# Open UART ports

---



# JTAG extracting Flash memory

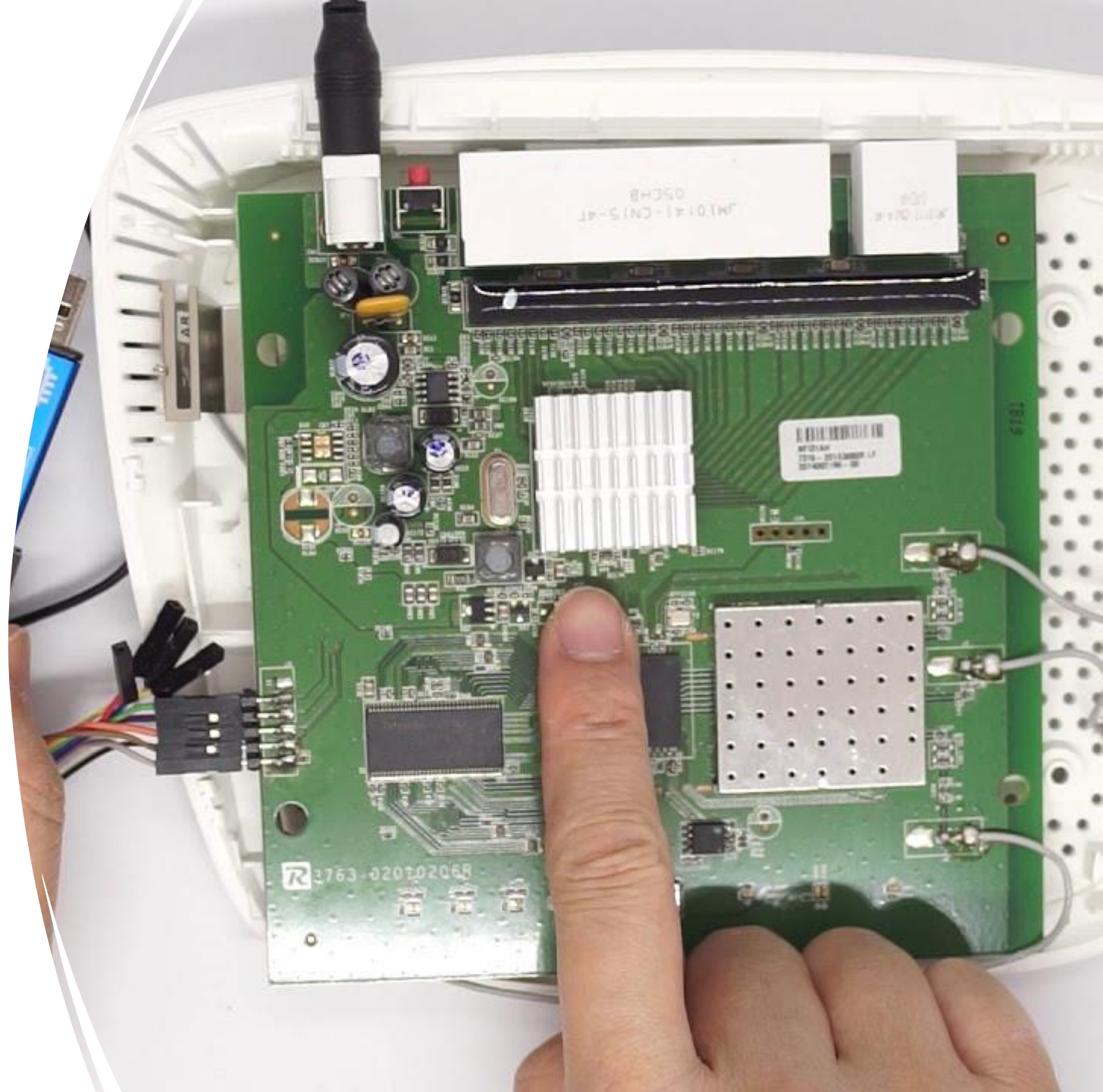
---



# JTAG extracting Flash memory

---

- [JTAG Explained \(finally!\): Why "IoT" Makers, Software Security Folks, and Device Manufacturers Should Care – Senrio](#)



# Firmware Penetration testing

- Binary Analysis
- Reverse Engineering
- Analysing different file system
- Sensitive key and certificates
- Firmware Modification

# Firmware Penetration testing

```
root@kali: ~/Downloads
File Edit View Search Terminal Help
root@kali:~/Downloads# binwalk -e TL-WR841nv14_EU_v9.1.4.16_up_boot\100319-rel57291\*.bin
DECIMAL      HEXADECIMAL    DESCRIPTION
-----
53952        0xd2c0         U-Boot version string, "U-Boot 1.1.3 (Mar 19 2018 - 15:36:42)"
66560        0x10400        LZMA compressed data, properties: 0x30, dictionary size: 8388608 bytes, uncompressed size: 2986732 bytes
1049088      0x100200        Squashfs filesystem, little endian, version 4.0, compression:xz, size: 2960369 bytes, 611 inodes, blocksiz
: 262144 bytes, created: 2018-03-19 07:55:53
root@kali:~/Downloads#
```

Extraction of .bin file

```
root@kali: ~/Downloads/squashfs-root
File Edit View Search Terminal Help
root@kali:~/Downloads/squashfs-root# ls -lh
total 44K
drwxr-xr-x. 2 root root 4.0K Mar 19 2018 bin
drwxr-xr-x. 5 root root 4.0K Mar 19 2018 dev
drwxr-xr-x. 5 root root 4.0K Mar 19 2018 etc
drwxr-xr-x. 3 root root 4.0K Mar 19 2018 lib
lrwxrwxrwx. 1 root root 11 Mar 19 2018 linuxrc -> bin/busybox
drwxr-xr-x. 2 root root 4.0K Mar 19 2018 mnt
drwxr-xr-x. 2 root root 4.0K Mar 19 2018 proc
drwxr-xr-x. 2 root root 4.0K Mar 19 2018/sbin
drwxr-xr-x. 2 root root 4.0K Mar 19 2018 sys
drwxr-xr-x. 4 root root 4.0K Mar 19 2018/usr
drwxr-xr-x. 2 root root 4.0K Mar 19 2018/var
drwxr-xr-x. 9 root root 4.0K Mar 19 2018/web
root@kali:~/Downloads/squashfs-root#
```

File system

```
# ls
bin      etc      home    lib      media    proc     sys      usr
dev      etc_ro  init    linuxrc  mnt     /sbin    tmp      var
# cd mnt
# ls
AWSCACertificate-ROOT.pem  gwlogger
Logs                       gwVersion.dat
MqttLib.tar.gz             gwWatchdog
SSLlib.tar.gz              fw_list.dat
bind.dat                   libloader.so
devicelistfile.dat         sc_list.dat
g_list.dat                 scenelistfile.dat
gwCloudStart               task.dat
gwConfig.dat               task_time.dat
gwDaemon                   timerlist.dat
gwDownload                 userDefence.dat
gwDnldInstrInstructions.txt userlist.dat
gwLoader                   zll
# cd Logs
# ls
gwDaemonLogs  gwLoaderLogs  gwMqttInfoLogs  gwWatchdogLogs
#
# cat gwMqttInfoLogs
112A1886:28/12/2018, 12:40:18: The Mqtt info of this gateway and its authentication details below:
112A1886:28/12/2018, 12:40:18: PWD: ADCC*****E50CC
112A1886:28/12/2018, 12:40:18: The Mqtt topics of this gateway below:
112A1886:28/12/2018, 12:40:18: Publish Topic: 112A1886/GwToApp
112A1886:28/12/2018, 12:40:18: Subscribe Topic: 112A1886/AppToGw
112A1886:28/12/2018, 12:40:18: HeartBeat Topic: 112A1886/HeartBeat
112A1886:28/12/2018, 12:40:18: Version Topic: 112A1886/GwVersionRequest
112A1886:28/12/2018, 12:40:18: Version Response Topic: 112A1886/GwVersionResponse
112A1886:28/12/2018, 12:40:18: App Download Topic: 112A1886/GwAppDownloadRequest
112A1886:28/12/2018, 12:40:18: Log Upload Topic: 112A1886/GwLogUploadRequest
daemon Version is: 1.0.6
#
```

Hardcoded MQTT credentials

# Radio Security Analysis

- Exploitation of communication protocols
  - BLE, Bluetooth, LORA, 6LoWPAN
- Sniffing Radio packets
- Jamming based attacks
- Modifying and replaying packets

# Exploiting BLE 4.0 Communication

bt2cap.cid == 0x0004

| No. | Time     | Source                  | Destination                    | Prot Length Info                                                |
|-----|----------|-------------------------|--------------------------------|-----------------------------------------------------------------|
| 10. | 22.8699. | SonyMobi_27:90:c8 (Xpe. | TexasIns_52:ff:b8 (Xpe.        | A. 14 Sent Find Information Request, Handles: 0x0042..0x0042    |
| 10. | 22.8876. | TexasIns_52:ff:b8 (Xpe. | SonyMobi_27:90:c8 (Xperia Z3+) | A. 15 Rcvd Find Information Response                            |
| 10. | 22.8985. | SonyMobi_27:90:c8 (Xpe. | TexasIns_52:ff:b8 (Xpe.        | A. 14 Sent Find Information Request, Handles: 0x0045..0x0045    |
| 10. | 22.9099. | TexasIns_52:ff:b8 (Xpe. | SonyMobi_27:90:c8 (Xperia Z3+) | A. 15 Rcvd Find Information Response                            |
| 10. | 22.9297. | SonyMobi_27:90:c8 (Xpe. | TexasIns_52:ff:b8 (Xpe.        | A. 14 Sent Find Information Request, Handles: 0x004a..0x004a    |
| 10. | 22.9306. | TexasIns_52:ff:b8 (Xpe. | SonyMobi_27:90:c8 (Xperia Z3+) | A. 15 Rcvd Find Information Response                            |
| 10. | 22.9428. | SonyMobi_27:90:c8 (Xpe. | TexasIns_52:ff:b8 (Xpe.        | A. 14 Sent Find Information Request, Handles: 0x004d..0x004d    |
| 10. | 22.9549. | TexasIns_52:ff:b8 (Xpe. | SonyMobi_27:90:c8 (Xperia Z3+) | A. 14 Rcvd Error Response - Attribute Not Found, Handle: 0x004d |
| 10. | 22.9855. | SonyMobi_27:90:c8 (Xpe. | TexasIns_52:ff:b8 (Xpe.        | A. 22 Sent Write Request, Handle: 0x0025                        |
| 10. | 23.0057. | TexasIns_52:ff:b8 (Xpe. | SonyMobi_27:90:c8 (Xperia Z3+) | A. 10 Rcvd Write Response                                       |
| 10. | 24.9997. | SonyMobi_27:90:c8 (Xpe. | TexasIns_52:ff:b8 (Xpe.        | A. 12 Sent Read Request, Handle: 0x002b                         |
| 10. | 25.1387. | TexasIns_52:ff:b8 (Xpe. | SonyMobi_27:90:c8 (Xperia Z3+) | A. 14 Rcvd Read Response                                        |
| 11. | 26.5583. | SonyMobi_27:90:c8 (Xpe. | TexasIns_52:ff:b8 (Xpe.        | A. 16 Sent Write Command, Handle: 0x002b                        |
| 11. | 26.6558. | SonyMobi_27:90:c8 (Xpe. | TexasIns_52:ff:b8 (Xpe.        | A. 12 Sent Read Request, Handle: 0x002b                         |
| 11. | 26.7476. | TexasIns_52:ff:b8 (Xpe. | SonyMobi_27:90:c8 (Xperia Z3+) | A. 14 Rcvd Read Response                                        |
| 11. | 26.9911. | TexasIns_52:ff:b8 (Xpe. | SonyMobi_27:90:c8 (Xperia Z3+) | A. 16 Rcvd Handle Value Notification, Handle: 0x002b            |
| 11. | 27.4089. | SonyMobi_27:90:c8 (Xpe. | TexasIns_52:ff:b8 (Xpe.        | A. 16 Sent Write Command, Handle: 0x002b                        |
| 11. | 27.4768. | TexasIns_52:ff:b8 (Xpe. | SonyMobi_27:90:c8 (Xperia Z3+) | A. 16 Rcvd Handle Value Notification, Handle: 0x002b            |

Frame 1109: 16 bytes on wire (128 bits), 16 bytes captured (128 bits) on interface  
Bluetooth  
Bluetooth HCI H4  
Bluetooth HCI ACL Packet  
Bluetooth L2CAP Protocol  
Length: 7  
CID: Attribute Protocol (0x0004)  
Bluetooth Attribute Protocol  
Opcode: Write Command (0x52)  
0... .. = Authentication Signature: False  
1... .. = Command: True  
01 0010 = Method: Write Request (0x12)  
Handle: 0x002b (Unknown)  
[UUID: Unknown (0xffd2)]  
Value: d0ffff

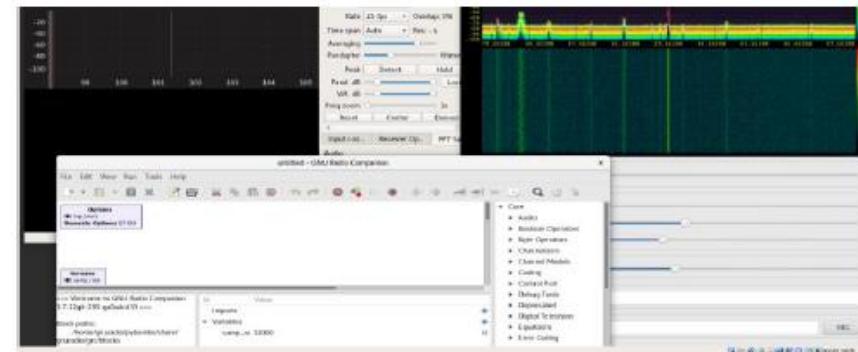
0000 02 40 00 0b 00 07 00 04 00 52 2b 00 10 ff ff ff 00 ..... R+

btsnoop\_hci.log

```
root@kali:~/Desktop# gatttool -i hci1 -b 8C:8B:83:52:FF:B8 --char-write-req -a 0x002b -n d0ff000ff
Characteristic value was written successfully
root@kali:~/Desktop#
```

```
root@kali:~# hcitool -i hci1 leinfo 8C:8B:83:52:FF:B8
Requesting information ...
Handle: 70 (0x0046)
LMP Version: 4.0 (0x6) LMP Subversion: 0x132
Manufacturer: Texas Instruments Inc. (13)
Features: 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00
```

# Analysis of radio signals using USRP



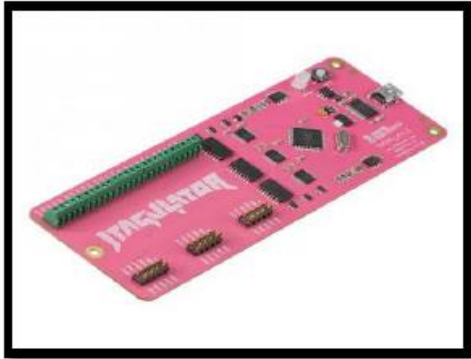
# Mobile, Web and Cloud Application Testing

- Web dashboards- XSS, IDOR, Injections
- .apk and .ios source code review
- Application reversing
- Hardcoded api keys
- Cloud Credentials like MQTT, CoAP, AWS...

# Software tools

| Hardware Level | Firmware Level | Radio Security |
|----------------|----------------|----------------|
| Baudrate.py    | Binwalk        | Gatttool       |
| Esptool        | Strings        | hcitool        |
| Flashrom       | IDAPro         | GNURadio       |
| Minicom        | Radare2        | Killerbee      |
| Screen         | Qumu           |                |
|                |                |                |

# Hardware tools



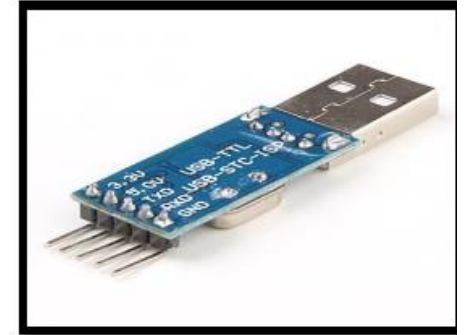
Jtagulator



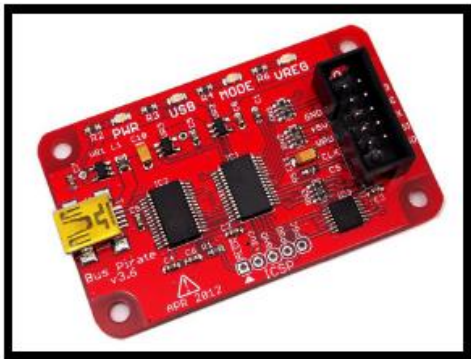
HackRF



Ubertooth



TTL-USB Converter



Bus Pirate



Zigbee Sniffer



Chip whisperer

A hand wearing a blue nitrile glove is shown touching a digital screen. The screen displays a blue background with a white line graph that has several red data points. The overall lighting is dim, with a strong blue hue from the screen's light.

# Assignment

Perform a pentest on the smart device!

# Example case in depth

---



# Attack methodologies

- Black box
- White box
- Grey box

# Ways to attack

- Port Scanning
- Man in the middle
- Brute force
- Dictionary attack
- USB Exploitation
- Firmware extraction
- Spoofing
- ...

# Threat modelling of the Vacuum cleaner system



Table I  
ASSETS OF THE ROBOT VACUUM CLEANER

| ID | Assets                   | Description                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Robot vacuum             | Has an Infrared Signal receiver, camera, bumper, charging sensors and various tools for cleaning. The robot will automatically return to the charging station once cleaning is done or battery is running low. Also includes a power switch, a voice prompt and a pause button.                                                                                                                                                 |
| 2  | Mobile application       | Available for Android and iOS. Used to control the robot remotely (or put it in auto cleaning mode) and also allows for taking a photo, monitor audio and recording. It also enables the user to find saved photos and videos and share them to social media such as facebook and twitter and can be set to give motion detection warnings. It also includes the possibility to perform a firmware upgrade and a factory reset. |
| 3  | Firmware                 | Movement of the robot and various camera features are controlled by the firmware.                                                                                                                                                                                                                                                                                                                                               |
| 4  | Remote                   | Contains power button, Auto cleaning button, Auto charging button, Spot cleaning button, Directional buttons, Pause button, Scheduling button, Normal- and Turbo Suction button.                                                                                                                                                                                                                                                |
| 5  | Charging/docking station | Has an Infrared Signal Transmitter to communicate with the Robot vacuum.                                                                                                                                                                                                                                                                                                                                                        |
| 6  | Internal device hardware | Not much is known about the hardware since the Robot vacuum has not been disassembled.                                                                                                                                                                                                                                                                                                                                          |

# Device Architecture Overview

- In order to get a better understanding of how the system (i.e. the vacuum and its assets) works before creating the architecture diagram,
- some use cases were created.

# Use case 1: User views camera feed live via the mobile application

1. User downloads and installs the application.
2. User creates an account with their e-mail.
3. User scans the QR-code on the back of the vacuum and gives it a name to bind it to the application (Note: Each robot vacuum can connect to at most 2 accounts and be controlled by at most 2 different devices at the same time).
4. User connects the robot vacuum to Wi-Fi by choosing the Smart Link option.
5. User navigates to the "Main console" page in the application and views the live feed.

Use case 2: User records video with the application during the day and views it later in the evening

1. to 4. steps are the same as in Use case 1.
5. User navigates to the "Main console" page in the application and presses the "record" button.
6. User later stops the recording and navigates to the "Personal file" page, selects the video that has been recorded and views it.

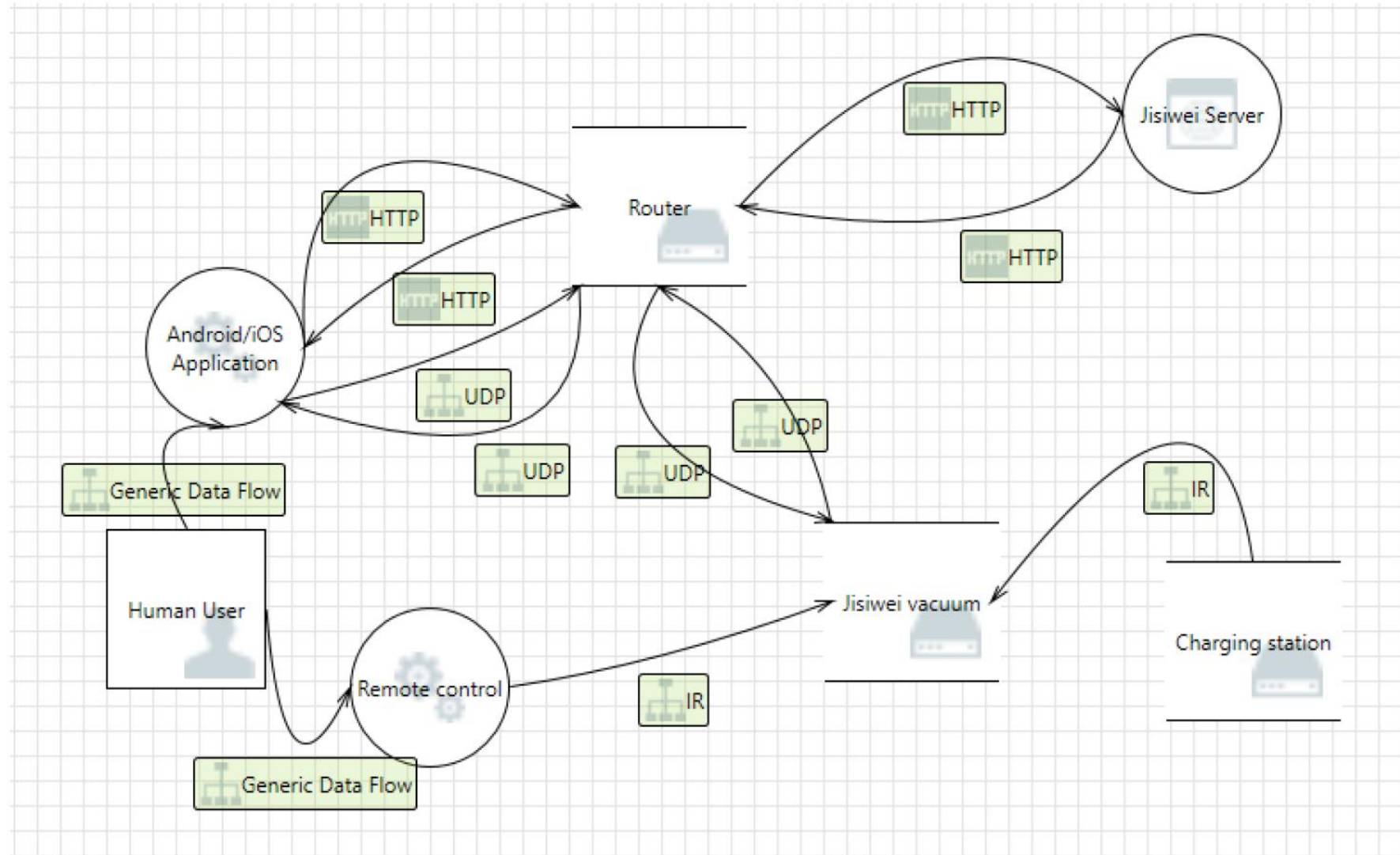
## Use case 3: User modifies personal information in the application

- 1) User opens the menu on the left side of the application.
- 2) From the menu user presses account name → personal information
- 3) User presses password and changes the password.

## Use case 4: User activates the motion detection feature while away during the weekend

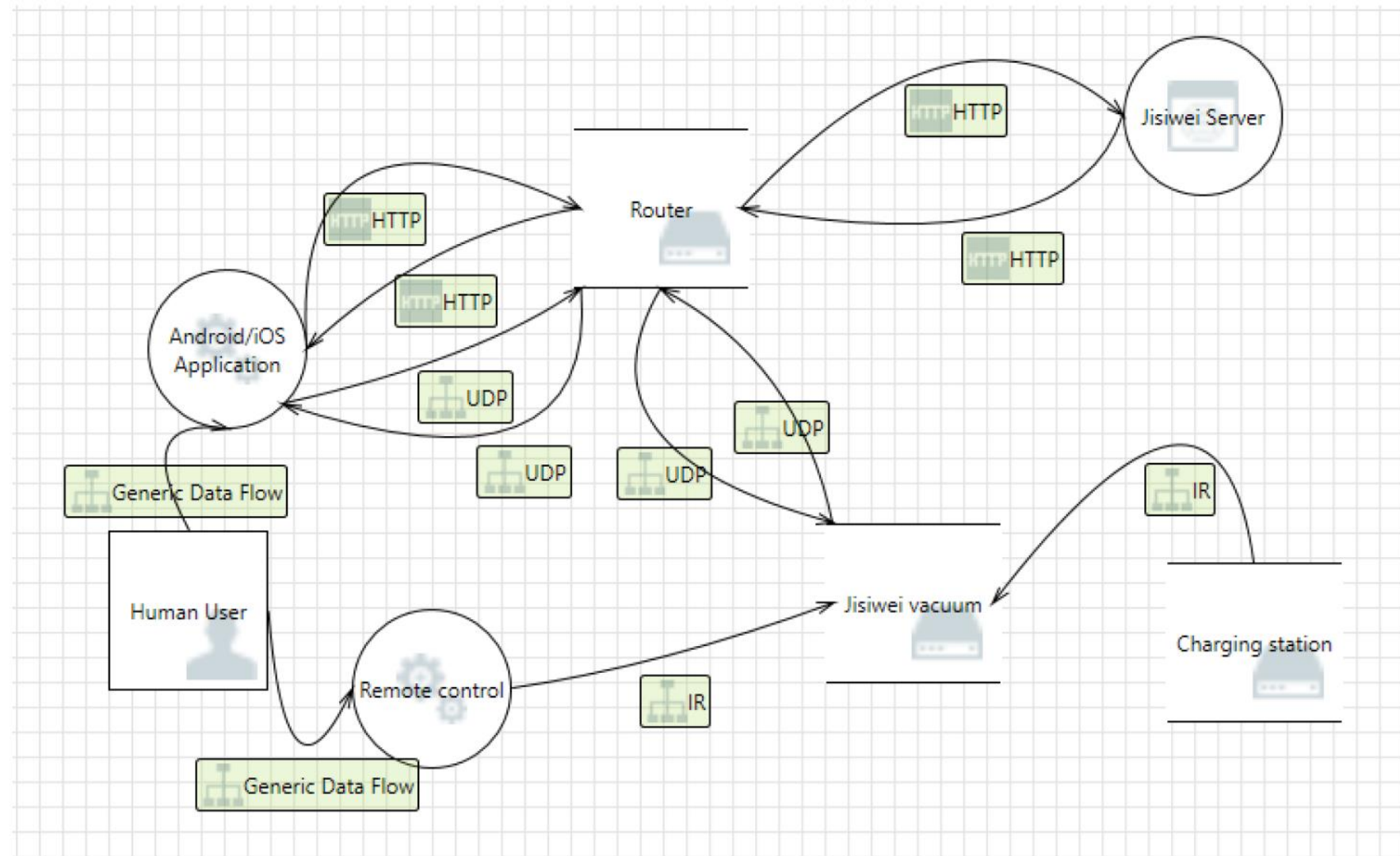
1. User presses "Motion Detective Warning" in the application.
2. User enables "Motion Detective Protection" chooses "low" motion detection sensitivity and presses "save".
3. A suspicious image occurs and the vacuum sends an alarm to the mobile device which will be shown in the "Main console" in the application.

# Device architecture overview



# Decomposing Device

- Communication between the mobile application and the server could be viewed and/or altered
- Communication between the mobile application and the vacuum could be viewed and/or altered
- Communication between the server and the vacuum could be viewed and/or altered



# Identifying Threats

- This table shows a selection of the threats, that were identified
- using the STRIDE mnemonic. The threats covered here are the
- ones that were deemed most likely to appear.

Table II  
THREATS IDENTIFIED WITH STRIDE

| Threat Category        | Threat                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Spoofing               | Attacker manages to get access to communication flows between the mobile application and the server by pretending to be the router.                                                                                                                                                                                                                                                                 |
| Tampering              | <ul style="list-style-type: none"><li>• Attacker reads from the eMMC memory, inserts malicious commands in files and writes the modified version back to the eMMC [9].</li><li>• Attacker modifies the APK (binaries) to make malicious mobile application [24].</li></ul>                                                                                                                          |
| Repudiation            |                                                                                                                                                                                                                                                                                                                                                                                                     |
| Information Disclosure | <ul style="list-style-type: none"><li>• Attacker gets access to the firmware during an upgrade with a man-in-the-middle attack [24].</li><li>• Attacker decompiles and analyzes the APK (binaries) for the Android application to find sensitive information [24].</li><li>• Attacker dumps potential secrets from the EEPROM (Electrically Erasable Programmable Read-Only Memory) [24].</li></ul> |
| Denial of Service      |                                                                                                                                                                                                                                                                                                                                                                                                     |
| Elevation of Privilege | <ul style="list-style-type: none"><li>• Attacker gets root access via an open SSH (port 22) or Telnet (port 23) [5].</li><li>• Attacker gets root access by plugging in to the UART [9].</li></ul>                                                                                                                                                                                                  |

# Documenting the Threats

The threats were documented by listing them with their respective attack techniques and countermeasures, which can be seen in table III.

Table III  
DOCUMENTATION OF THE THREATS

| Threat                                                                                                                              | Attack Techniques                                                                                                                                                                                                               | Countermeasures                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Attacker manages to get access to communication flows between the mobile application and the server by pretending to be the router. | Attacker could use proxy tools such as mitmproxy or Burp Suite in combination with ARP spoofing/poisoning.                                                                                                                      | Use HTTPS protocol for communication.                                    |
| Attacker reads from the eMMC memory, inserts malicious commands in files and writes the modified version back to the eMMC.          | Attacker could use an SD card reader/writer to modify the contents of the eMMC <sup>14</sup> .                                                                                                                                  | Assemble the eMMC in such a way that it is hard to physically access it. |
| Attacker decompiles and analyzes the APK (binaries) for the Android application to find sensitive information.                      | The APK (binary) file can be downloaded from a third party client (such as APKpure.com) and a tool such as Enjarify could be used to generate Java bytecode which could be searched to look for potential vulnerabilities [24]. | Attempts could be made to keep the APK from being available online.      |
| Attacker modifies the APK (binaries) to make malicious mobile application.                                                          | By decompiling the APK to Java bytecode, an attacker could make changes in the Java bytecode and then compile it back to APK.                                                                                                   | Attempts could be made to keep the APK from being available online.      |

# Documenting the Threats

The threats were documented by listing them with their respective attack techniques and countermeasures, which can be seen in table III.

|                                                                                         |                                                                                                                                                          |                                                                                                           |
|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Attacker gets access to the firmware during an upgrade with a man-in-the-middle attack. | Attacker could use proxy tools such as mitmproxy or Burp Suite in combination with ARP spoofing/poisoning.                                               | The firmware is sent encrypted to the vacuum cleaner.                                                     |
| Attacker dumps potential secrets from the EEPROM.                                       | By using a SOIC clip an attacker could read from the EEPROM [24].                                                                                        | Do not store sensitive information or secrets on the EEPROM [24].                                         |
| Attacker gets root access via an open SSH (port 22) or Telnet (port 23).                | Attacker could use a password cracking tool like Hydra or Medusa and attempt dictionary attacks to get access to the SSH or Telnet console.              | Create an iptables firewall rule that only accept traffic to port 22 or 23 from certain IP addresses [5]. |
| Attacker gets root access by plugging in to the UART.                                   | By using a multimeter, the attacker identifies the four UART pins and uses a tool such as Attify Badge to get shell access to the UART-based shell [24]. | The UART pins are blocked by another chip to prevent access [24].                                         |

## Rating the threats

Table IV  
RATING OF THE THREATS USING DREAD

| Threat                                                                                                                                       | D | R | E | A | D | Risk score |
|----------------------------------------------------------------------------------------------------------------------------------------------|---|---|---|---|---|------------|
| Attacker gets access to log-in credentials and users personal information by proxying traffic between the mobile application and the server. | 3 | 3 | 3 | 3 | 3 | 15         |
| Attacker gets root access via an open SSH (port 22) or Telnet (port 23).                                                                     | 3 | 3 | 3 | 1 | 3 | 13         |
| Attacker gets root access by plugging in to the UART.                                                                                        | 3 | 3 | 3 | 1 | 2 | 12         |
| Attacker reads from the eMMC memory, inserts malicious commands in files and writes the modified version back to the eMMC.                   | 3 | 3 | 2 | 1 | 2 | 11         |
| Attacker gets access to the firmware during an upgrade with a MITM attack.                                                                   | 2 | 2 | 2 | 2 | 3 | 11         |
| Attacker decompiles and analyzes the APK (binaries) to find sensitive information.                                                           | 1 | 3 | 2 | 2 | 2 | 10         |
| Attacker dumps potential secrets from the EEPROM.                                                                                            | 1 | 3 | 2 | 1 | 2 | 9          |
| Attacker modifies the APK (binaries) to make malicious mobile application.                                                                   | 3 | 1 | 1 | 2 | 1 | 8          |

# Next step Penetration Testing Setup

The materials being used in the penetration testing of the device are the following:

- Jisiwei i3 robot vacuum cleaner
- Netgear Router
- PC with Kali Linux in a VM
- Samsung S5 Android smart phone
- Iphone 6s iOS smart phone

# Used software

- Kali Linux Distribution
- mitmproxy
- Wireshark
- ARPspooof (dsniff)
- Hydra

# Pentest MITM

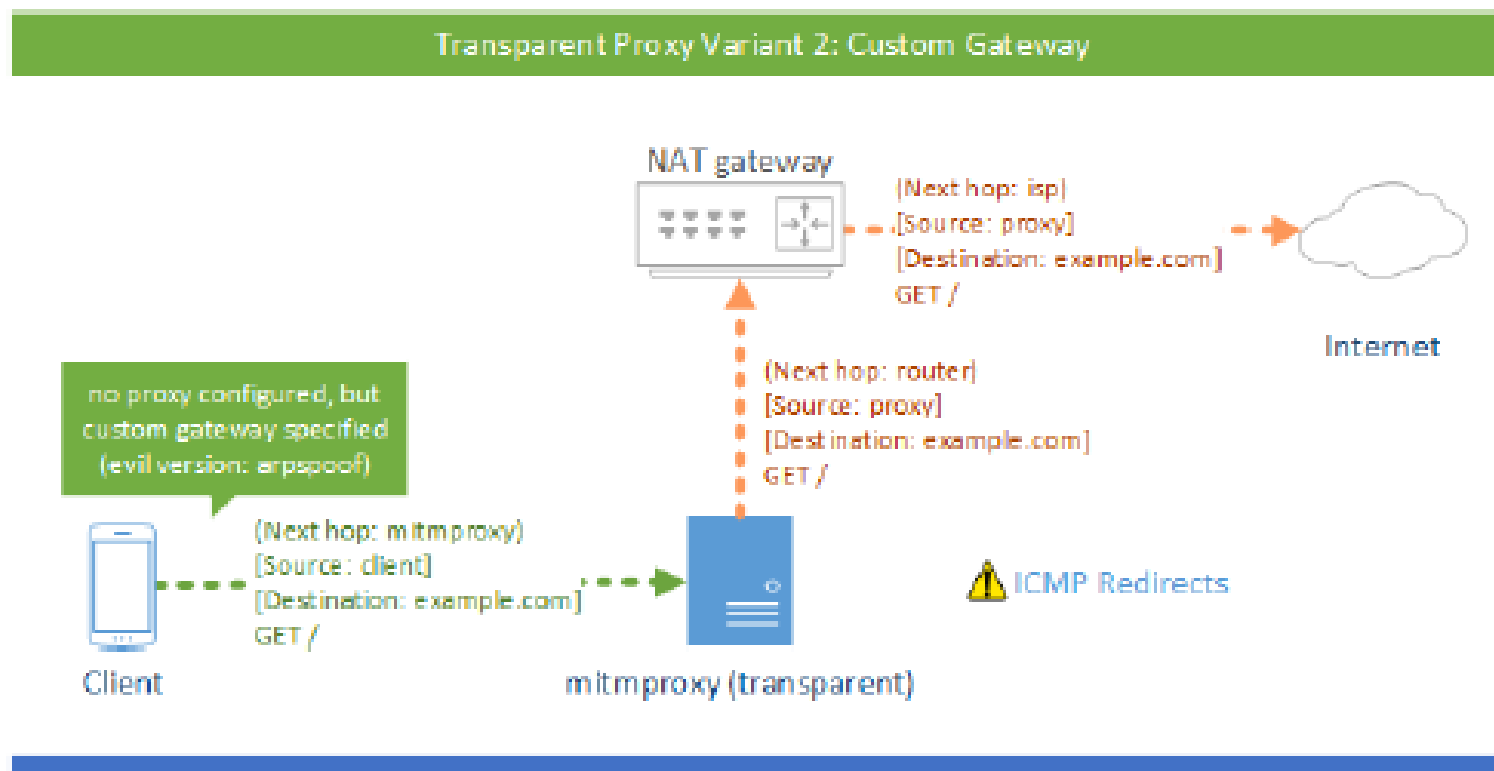


Figure 3. mitmproxy configured for transparent mode<sup>19</sup>.

# Results

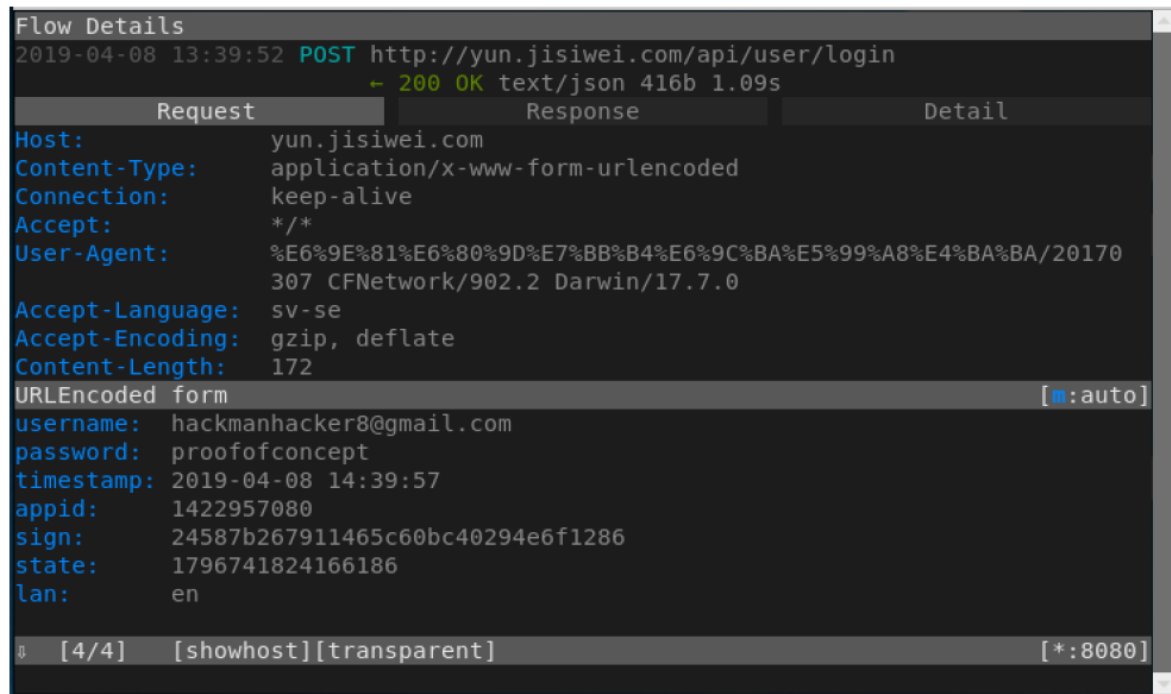


Figure 4. Login request sent to server from application. Packet captured with mitmproxy. As can be seen in the picture the username and password is available in plain text. Also notice the lack of HTTPS.

A serious flaw was discovered through the MITM attack:

- All of the communication between the mobile application and the server is sent unencrypted over HTTP.
- This allows an attacker on the same network to be able to read things such as username and password and all the data connected to the user account.
- Since everything is being sent in plain text there is no need for the attacker to do any other procedure such as trying to run it through some kind of decrypting tools to try to brute force hashes.
- The actual login request that was captured can be seen in figure 4.

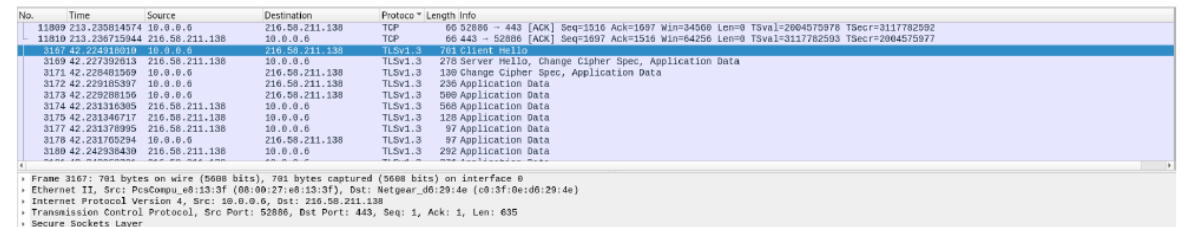


Figure 5. The firmware being sent from the server using HTTP over TLS version 1.3 (HTTPS), seen in the program Wireshark. In the info field to the right the firmware corresponds to what is named Application Data.

# Other PenTesting Tasks

- Port scanning
- Telnet port password cracking
- HTTP server password cracking
- QR code generation